Deusto



Desarrollo de GUI con Swing

Más allá del ejemplo básico



The Swing Tutorial

- Documentación de referencia
  - The Swing Tutorial
    - Descripción del uso de cada componente con ejemplos
    - Distribución de componentes en contenedores (Layout)
    - Gestión de eventos (Listeners)
    - Javadoc API - Desktop Module
  - Índice visual de los principales componentes



Contenedores de alto nivel

- Son el elemento principal de cualquier GUI Swing:
  - JFrame: representa una ventana principal.
  - JDialog: es un cuadro de diálogo (temporal) que suele depender de una ventana principal
  - JOptionPane: clase de utilidad para la creación de cuadros de diálogo estándar preconfigurados.



JFrame

- Métodos más utilizados:
  - setVisible(): muestra oculta el JFrame.
  - setTitle(): modifica el título.
  - setSize(): modifica el tamaño.
  - getContentPane(): obtiene el panel contenedor principal.
  - setLocation() / setLocationRelativeTo(): establece la posición en la patalla.
  - setDefaultCloseOperation(): define el comportamiento al cerrar el JFrame.
  - setIconImage(): modifica el icono del JFrame.
  - setJMenuBar(): establece una barra de menús.



JPanel

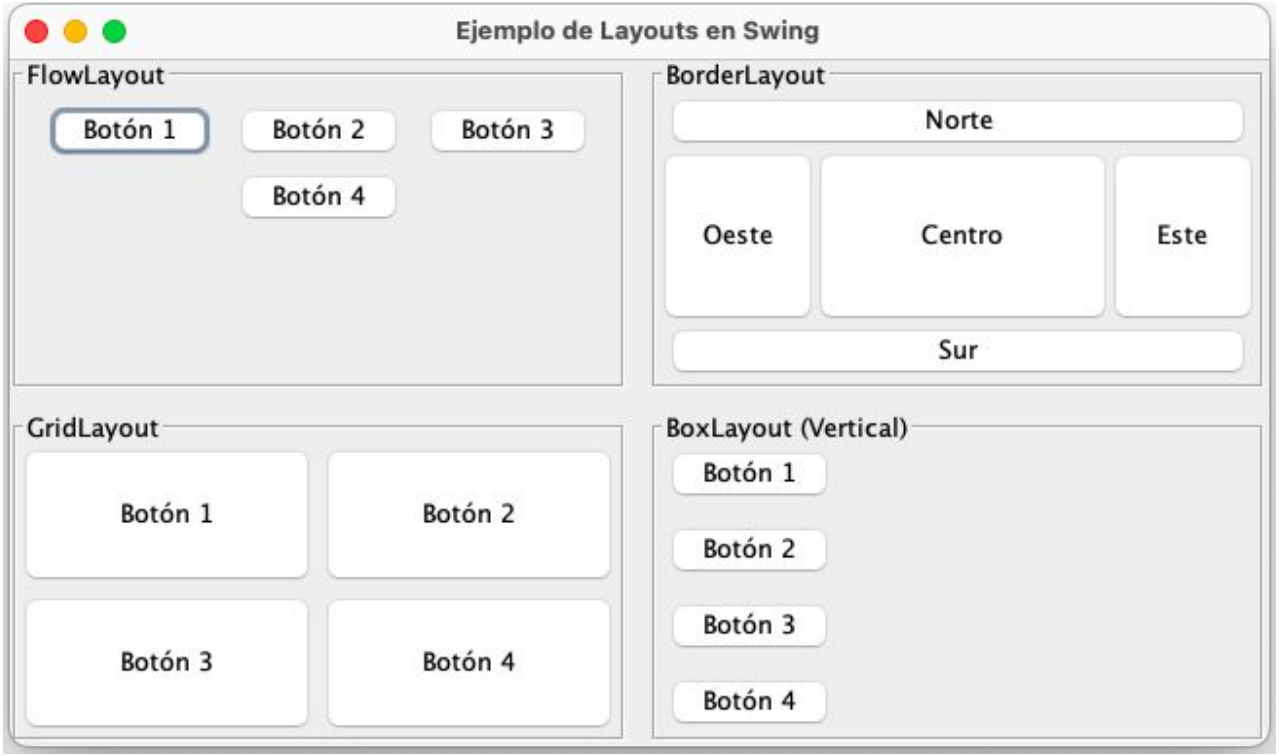
- Representa un panel rectangular que se utiliza para distribuir otro tipo de componentes en la GUI.
- La interacción habitual con un JPanel es la configuración de un Layout Manager y la adición de componentes.
- Otra operación habitual es decorar un JPanel con un borde:
  - setBorder()
  - *How to use Borders (Oracle, 2022)*



Layout Managers

- El layout define la manera en la que los componentes se distribuyen en un contenedor. *A Visual Guide to Layout Managers*
- ***El Layout Manager decide el tamaño y posición de los componentes.***
- Existen multiples Layout Managers pero los más utilizados son:
  - BorderLayout
  - GridLayout
  - FlowLayout (layout por defecto)
  - BoxLayout

Ejemplos de Layouts (swing-p1)





Componentes

- Los componentes visuales se definen con 3 elementos
  - **Data Model**: datos que visualiza/representa (TableModel)
    - **State Model**: información genérica del estado (ej.: botón pulsado)
  - **Look & Render**: aspecto visual.
  - **Events & Listeners**: comportamiento ante una interacción.
- El comportamiento de los componentes se basa en una versión adaptada del patrón MVC (ver referencia [oracle.com]).



Gestión de eventos

- Las interacciones con componentes de interfaz gráfica generan eventos (click de botón, click de ratón, pulsar una tecla, seleccionar un elemento de una lista,…).
- En la gestión de un evento participan dos elementos:
  - **Source**: componente que genera el evento.
  - **Listener**: funcionalidad que da respuesta al evento.
    - **Los listeners son interfaces** cuyo comportamiento debe ser definido de manera explícita es cada caso.



Listeners y Componentes

- Los listeners se registran en los componentes
  - JButton, JRadioButton, JComboBox - `addActionListener()`
  - JCheckBox - `addChangeListener()`
  - JTextField - `getDocument().addDocumentListener()`
  - JList - `addListSelectionListener()`
  - TableModel - `addTableModelListener()`
- 👉 **Lista completa de eventos para cada componente**



👉 JComponent

- Es la base para todos los componentes
  - `setVisible()`: muestra/oculta el componente.
  - `setEnabled()`: activa/desactiva el componente.
  - `setPreferredSize()`: define el tamaño por defecto.
  - `setForeground()`: define el color principal (color de la fuente).
  - `setBackground()`: define el color de fondo
  - `setBorder()`: define el borde.
  - `setFont()`: define la fuente.



Botón y botones de selección

- 👉 JButton
  - Además de texto puede contener imágenes (ImageIcon)
  - `addActionListener()`: ActionListener - `actionPerformed()`
  - `doClick()`: fuerza la invocación del método del ActionListener.
- 👉 JCheckBox
  - `getSelected()`: devuelve boolean con el estado.
  - `addItemListener()`: ItemListener - `itemStateChanged()`
- 👉 JRadioButton
  - Pueden agruparse en un ButtonGroup
  - `addActionListener()`: ActionListener - `actionPerformed()`



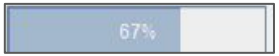
Campos de texto

- 👉 JTextField
  - Evento generado al pulsar enter:
    - `addActionListener()`: ActionListener - `actionPerformed()`
  - Evento generado al modificar el texto (tecleando):
    - `getDocument().addDocumentListener()`: DocumentListener
- 👉 JPasswordField: campo de texto para contraseñas.
- 👉 JFormattedTextField: campo de texto con formato predefinido (ejemplo fecha).
- 👉 JTextArea: campo de texto que admite varias filas.
- 👉 JTextPane: admite diferentes estilos en el texto e imágenes.

Otros componentes

- 👉 JSpinner
  - Selección de una lista, rango de números o fechas.
  - `addChangeListener()`: ChangeListener



- 👉 JSlider 
  - Selección de un valor numérico en una escala.
  - `addChangeListener()`: ChangeListener
- 👉 JProgressBar 

  - Progreso entre un valor mínimo y máximo. Diferencia el valor del texto mostrado.
  - `setValue()`: modifica el valor
  - `setStringPainted()`: muestra/oculta el % de progreso



Lista seleccionable

- 👉 JList
  - Lista de selección individual o múltiple (`setSelectionMode()`).
  - El texto de cada ítem es el `toString()` del tipo de objeto del componente.
  - Modelo de datos: DefaultListModel
  - Personalización del renderizado: ListCellRenderer
  - `setSelectedIndex()`: define el ítem seleccionado.
  - `getSelectedValue()`: devuelve el ítem seleccionado.
  - Evento de selección en la lista:
    - `addListSelectionListener()`: ListSelectionListener - `valueChanged()`



Lista desplegable

- 👉 JComboBox
  - El texto de cada ítem es el `toString()` del tipo de objeto del componente.
  - Modelo de datos: DefaultComboBoxModel
  - Personalización del renderizado: ListCellRenderer
  - `setEditable()`: determina si la lista permite edición o no.
  - `setSelectedIndex()`: define el ítem seleccionado.
  - `getSelectedItem()`: devuelve el ítem seleccionado.
- La gestión del cambio del elemento seleccionado se puede gestionar de 2 formas:
  - `addActionListener()` - ActionListener: Se genera un único evento.
  - `addItemListener()` - ItemListener: Se generan 2 eventos (DESELECTED + SELECTED).



Menús

- 👉 JMenuBar: Barra de menú principal
  - `setJMenuBar()`: añade la barra de menú principal a un JFrame.
- 👉 JMenu: Representa la raíz de un menú
- 👉 JMenuItem
  - Elemento de menú.
  - `setMnemonic()`: añade teclas de acceso rápido
  - Puede contener iconos (ImageIcon), separadores, submenús y botones de selección (JRadioButtonMenuItem y JCheckBoxMenuItem).



Cuadros y ventanas de diálogo

- 👉 JDialog
  - Diálogo/Pop-up
- 👉 JOptionPane
  - Crea diálogos predefinidos: Message, Confirm, Input, Option
- 👉 JFileChooser
  - Diálogo para seleccionar un archivo del sistema de ficheros.
  - `addChoosableFileFilter()`: define un filtro de extensiones aceptadas.
  - `showSaveDialog()`: para crear un nuevo fichero.
  - `showOpenDialog()`: para abrir un fichero existente.



Tablas

- 👉 JTable
  - Muestra información en formato tabla (ej. una lista de objetos de una clase).
  - Modelo de datos: DefaultTableModel
  - Personalización del renderizado de las celdas: TableCellRenderer
  - Personalización de la edición del contenido de las celdas: TableCellEditor
  - Evento de selección de una fila
    - `getSelectionModel().addListSelectionListener()`
  - Evento de modificación de los datos de la tabla
    - `tableModel.addTableModelListener()`



Métodos útiles de JTable

- `Object getValueAt(int row, int column)`: valor de una celda
- `getTableHeader()`: información de la cabecera
- `getColumnModel()`: información de las columnas
- `getSelectionModel()` - información del modelo de selección de filas
- `setSelectionMode(int mode)`: define el modo de selección de la tabla
- `int columnAtPoint(Point point)`: columna dado un punto
- `int rowAtPoint(Point point)`:fila dado un punto
- `int[] getSelectedRows()`: índices de filas seleccionadas
- `boolean isRowSelected(int row)`: indica si la fila está seleccionada



Modelo de datos personalizado

- Si la estructura de datos que muestra un JTable es compleja, se crea un modelo de datos con una nueva clase que incorpore un atributo con la estructura de datos (ej. una lista o mapa) y herede de DefaultTableModel o AbstractTableModel y redefina algunos métodos:
  - `int getRowCount()`: número de filas
  - `int getColumnCount()`: número de columnas
  - `String getColumnName(int col)`: nombre de una columna
  - `Object getValueAt(int row, int col)`: valor de una celda
  - `boolean isCellEditable(int row, int col)`: ¿se puede editar?

Renderizado de celdas de una tabla

- El render por defecto es un JLabel con el String del método toString() del objeto.
- 👉 TableCellRenderer: permite personalizar el renderizado de las celdas.
- Normalmente se hereda de la clase DefaultTableCellRenderer y se redefine el método getTableCellRendererComponent().
- El método se invoca para cada una de las celdas de la tabla.
- `getTableCellRendererComponent(table, value, isSelected, hasFocus, row, col)`
  - El retorno es un Componente (JLabel, JButton, ImageIcon, JCheckBox, …).
  - **value** es el dato que se renderiza en la celda actual.
  - **row** y **col** la fila/columna de la celda actual
  - **isSelected** indica si la celda está seleccionada.
  - **hasFocus** indica si la celda tiene el “foco”.



Edición de celdas de una tabla

- Parecido al renderizado. Por defecto, la edición se basa en un JTextField.
- 👉 TableCellEditor: permite personalizar la edición de las celdas.
- Normalmente se hereda de la clase DefaultCellEditor y se redefine el método getTableCellEditorComponent().
- `getTableCellEditorComponent(table, value, isSelected, row, col)`
  - El retorno es un Componente (JTextField, JCheckBox y JComboBox)
  - **value** es el dato de la celda que se están editando
  - **row** y **col** representan la fila/columna de la celda que se está editando
  - **isSelected** indica si la celda está seleccionada.



Actualización de los datos de una tabla

- La modificación de los datos en una tabla crea eventos TableModelEvent.
  - El evento identifica el tipo de modificación, columna, primera y última fila.
  - Los tipos de modificación son UPDATE (0), INSERT (1), DELETE (-1).
- Los cambios en el modelo de datos se procesan con un TableModelListener
  - `dataModel.addTableModelListener()`
  - 👉 TableModelListener - `tableChanged(TableModelEvent e)`



Expresiones lambda en Java

- Una función lambda en Java es lo complementario a una interface: **implementa una interfaz con un único método.**

  (/* parámetros */ **) -> {** /* código java */ **}**

- Usos más habituales:
  - Parámetro del método **forEach()** para recorrer y procesar Colecciones.
  - Implementar interfaces con un único método: **Runnable**, **ActionListener**, **TableCellRenderer**, **Comparable**, ….



Eventos de ratón

- Las interacciones con el ratón se representan mediante un MouseEvent, cada evento contiene la siguiente info:
  - `int getButton()` - botón pulsado
  - `Point getPoint()` - posición relativa al componente origen
- Estos eventos de ratón se pueden procesar con 3 interfaces
  - MouseListener
  - MouseMotionListener
  - MouseWheelListener



Listeners de eventos de ratón (general)

- MouseListener
  - `mouseEntered()` / `mouseExited()`
  - `mouseClicked()`
  - `mousePressed()` / `mouseReleased()`
  - `mouseWheelMoved()`
  - MouseAdapter clase base para crear listeners.



Listeners de eventos de ratón (movimientos)

- MouseMotionListener
  - `mouseMoved()`
  - `mouseDragged()`
  - MouseMotionAdapter clase base para crear listeners.



Eventos de teclado

- El componente que tiene el foco genera eventos KeyEvent al pulsar teclas.
  - `char getKeyChar()` - caracter pulsado
  - `int getKeyCode()` - código del carácter pulsado (constantes estáticas)
  - `boolean isActionKey()` - ¿tecla es de acción/función?
  - `boolean isAltDown()` - ¿se ha pulsado ALT?
  - `boolean isAltGraphDown()` - ¿se ha pulsado ALT GR?
  - `boolean isControlDown()` - ¿se ha pulsado CTRL?
  - `boolean isShiftDown()` - ¿se ha pulsado SHIFT?

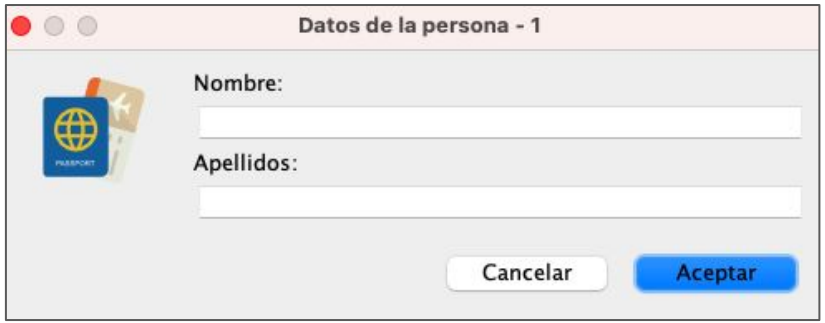


Listener de eventos de teclado

- KeyListener
  - `keyPressed(KeyEvent e)` - pulsar tecla
  - `keyReleased(KeyEvent e)` - soltar tecla
  - `keyTyped(KeyEvent e)` - escribir caracter en componente de texto
  - KeyAdapter clase base para crear listeners.

Ejemplos avanzados de Swing

- Crear un JOptionPane usando componentes personalizados
  - El atributo **message** y **option** es un Objeto (también puede ser un array).
    - El renderizado “por defecto” es un JLabel y como texto toString().
    - Si se usa un JComponent renderiza gráficamente.



Se crea un array JComponent con dos **JLabel** y dos **JTextField** y se pasa como parámetro message al crear el **JOptionPane**.

Ejemplos avanzados de Swing

- Actualizar la GUI al modificar la selección de un JComboBox
  - Se define un ActionListener con el comportamiento.



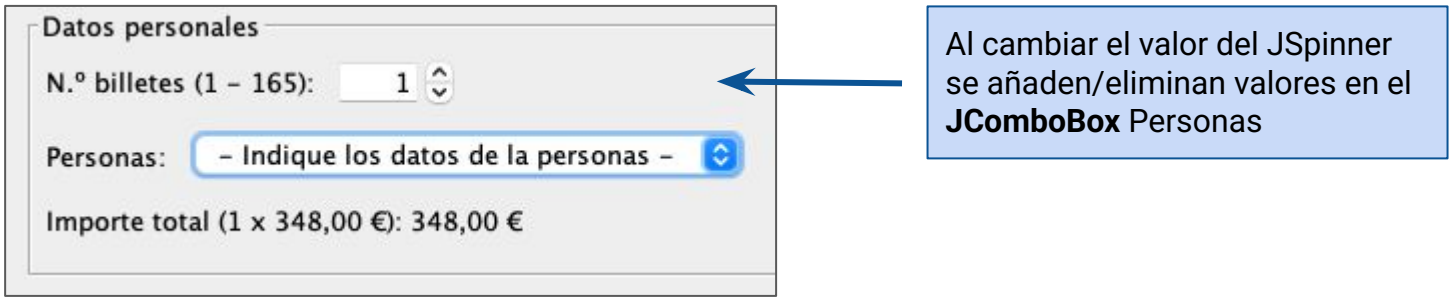
Al seleccionar un valor de Origen, se actualizan los valores del **JComboBox** Destino.

Al seleccionar un valor en Destino se actualiza la tabla central.

AEROLÍNEA	VUELO	ORIGEN	DESTINO	DURACIÓN	PRECIO	RESERVAS	ASIENTOS LIBRES	DISPONIBILIDAD	CHECK-IN	RESERVAR
	DL2742	BCN	JFK	491 m.	1079,00 €	0	400	100,0 %	Check-In	Reservar
	AA5406	BCN	JFK	492 m.	1180,00 €	0	216	100,0 %	Check-In	Reservar
	IB4610	BCN	JFK	493 m.	1204,00 €	0	216	100,0 %	Check-In	Reservar
	BA5798	BCN	JFK	491 m.	1219,00 €	0	216	100,0 %	Check-In	Reservar
	AF2604	BCN	JFK	499 m.	1231,00 €	0	400	100,0 %	Check-In	Reservar

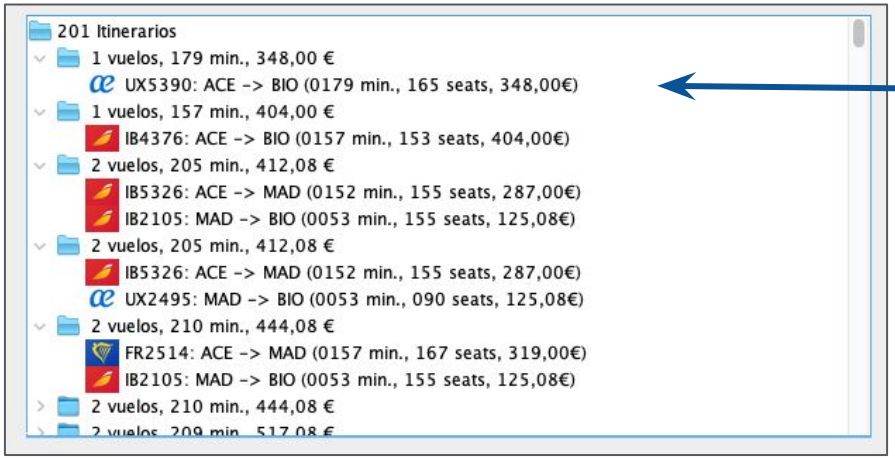
Ejemplos avanzados de Swing

- Actualizar los valores de un JComboBox al modificar un JSpinner
  - Se define un ChangeListener y se obtiene el valor del modelo de datos del JSPinner.



Ejemplos avanzados de Swing

- Personalizar el renderizado de un JTree
  - Se puede usar un DefaultTreeCellRenderer y redefinir el método getTreeCellRendererComponent()



Si el nodo no es hoja se renderiza "por defecto" (una carpeta).

Si el nodo es hoja, se renderiza como un **JLabel** con imagen y texto.



Ejemplos avanzados de Swing

- Uso de un botón para renderizar la celda de una tabla
  - Se crear una clase que es a la vez Renderer y Editor:
    - `extends AbstractCellEditor implements TableCellEditor, TableCellRenderer`

AEROLÍNEA	VUELO	ORIGEN	DESTINO	DURACIÓN	PRECIO	RESERVAS	ASIENTOS LIBRES	DISPONIBILIDAD	CHECK-IN	RESERVAR
	DL2742	BCN	JFK	491 m.	1079,00 €	0	400	100,0 %	Check-In	Reservar
	AA5406	BCN	JFK	492 m.	1180,00 €	0	216	100,0 %	Check-In	Reservar
	IB4610	BCN	JFK	493 m.	1204,00 €	0	216	100,0 %	Check-In	Reservar
	BA5798	BCN	JFK	491 m.	1219,00 €	0	216	100,0 %	Check-In	Reservar
	AF2604	BCN	JFK	499 m.	1231,00 €	0	400	100,0 %	Check-In	Reservar

Al pulsar en **Check-in** o **Reservar** se abre un cuadro de diálogo

Bibliografía

- MIT. (2018). A Visual Guide to Swing Components. 👉 URL
- Oracle. (2022a). A Visual Guide to Layout Managers. 👉 URL
- Oracle. (2022b). Creating a GUI With Swing. 👉 URL
- Oracle. (2022c). How to Write a Key Listener. 👉 URL
- Oracle. (2022d). How to Write a Mouse Listener. 👉 URL
- Oracle. (2022e). Laying Out Components Within a Container. 👉 URL
- Oracle. (2022f). Using Swing Components. 👉 URL
- Oracle. (2022g). Writing Event Listeners. 👉 URL
- Oracle. (2023a). A Swing Architecture Overview. 👉 URL
- Oracle. (2023b). Java Development Kit Version 21 API Specification. 👉 URL